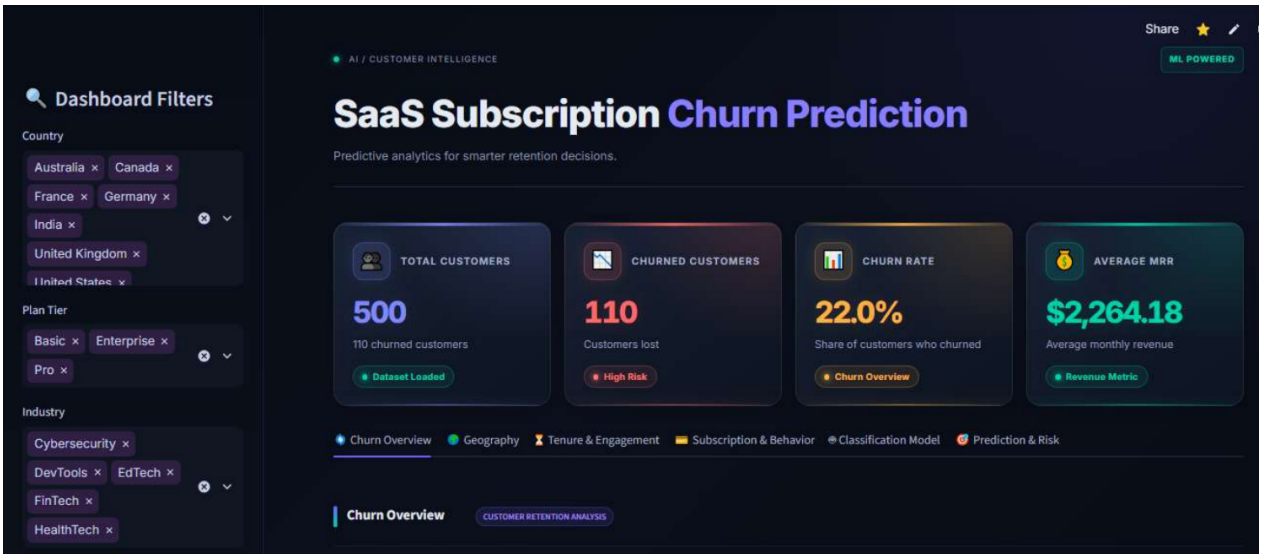
INTERNSHIP PROJECT REPORT

SaaS Subscription Churn Prediction

Machine Learning and Interactive Analytics



The Streamlit dashboard built in this project

Submitted to	Author	Submission date
Naviotech Solution Pvt. Ltd.	**Pratham Rangoonwala**	**25/10/2026**

Components: data cleaning and exploratory analysis notebooks, a Gradient Boosting churn classifier, and a six-tab Streamlit dashboard, built on the Kaggle RavenStack SaaS Subscription & Churn Analytics dataset.

Abstract

This report documents an internship project that analyses customer churn for a subscription-software (SaaS) business and exposes the results through an interactive dashboard. The work uses the five-table RavenStack dataset from Kaggle (500 accounts, 110 of which are labelled as churned, a 22.0% churn rate). Raw tables were cleaned, aggregated to one row per account, and explored by plan, country, industry, tenure and subscription behaviour. A churn classifier was then built from 19 account-level features that are computed only from records dated before a per-account prediction date. Three candidate models were compared on a validation set; Gradient Boosting was selected, its decision threshold (0.40) was chosen on the validation set, and the final model was scored once on a held-out test set of 100 accounts: accuracy 0.900, precision 0.800, recall 0.727, F1 0.762 and ROC-AUC 0.876. A six-tab Streamlit application presents the exploratory findings, model results and a per-account risk lookup.

The report also records what the project does not establish. The test set contains only 22 churned accounts, so the metrics carry wide uncertainty; some churned accounts have missing behavioural features, and this missingness appears to help the model; and the dashboard scores accounts with whole-history features that differ from the training windows. These points are set out in Section 9 so that the reported numbers can be read in context. All exploratory findings are associations in a dataset described as synthetic; none is a causal claim.

Keywords: customer churn, SaaS analytics, gradient boosting, classification, Streamlit, data visualisation.

Contents

1 Introduction
2 Dataset
3 System Architecture and Workflow
4 Data Preparation and Feature Engineering
5 Exploratory Analysis
6 Modelling Methodology
7 Results
8 Streamlit Dashboard
9 Limitations and Validity
10 Conclusion and Future Work
References
A Appendix A - Verification Status

1 Introduction

1.1 Background

Subscription businesses lose revenue whenever an account cancels. Retention teams therefore want to know which customer segments leave more often and, where possible, which accounts are at risk before they leave. This project, carried out during an internship at Naviotech Solution Pvt. Ltd., builds that analysis end to end on a public SaaS dataset: cleaning, exploration, a predictive model, and an interactive dashboard for non-technical readers.

1.2 Objectives

- Prepare the five RavenStack tables for analysis and combine them into one row per account.
- Describe how churn varies across plan tier, geography, industry, tenure and subscription behaviour.
- Train and compare classification models for account-level churn, choosing the model and decision threshold without using the test set.
- Present the findings, model results and a per-account risk lookup in a Streamlit dashboard.

1.3 Scope and Boundaries

The report documents the project as built. It does not re-design the model or the application. Four boundaries apply throughout:

- **Dataset.** Findings describe this dataset only. Public descriptions characterise it as synthetic [2], so results should not be generalised to real customers.
- **Association, not causation.** Differences in churn rates between groups are descriptive.
- **No operational claims.** The project does not claim production deployment, real-time scoring, alerting or monitoring.
- **Metrics are labelled by split.** Validation metrics (model selection) and test metrics (final evaluation) are reported separately and are not interchangeable.

2 Dataset

2.1 Source

The data is the RavenStack SaaS Subscription & Churn Analytics dataset published on Kaggle by River @ Rivalytics [1]. It consists of five related CSV tables describing accounts, their subscriptions, feature usage, support tickets and churn events.

2.2 Tables and Relationships

Table 1. Source tables (cleaned versions).

Table	Rows x columns	Content	Join key
accounts	500 x 10	Industry, country, signup date, referral source, plan tier, seats, trial flag, churn_flag	account_id
subscriptions	5,000 x 14	Start/end dates, MRR/ARR, plan tier, upgrade, downgrade and auto-renew flags	account_id
feature_usage	25,000 x 8	Usage date, feature name, usage count, duration, errors, beta-feature flag	subscription_id

Table	Rows x columns	Content	Join key
support_tickets	2,000 x 9	Submitted/closed time, resolution time, priority, first response time, satisfaction, escalation	account_id
churn_events	600 x 9	Churn date, reason code, refund amount, reactivation flag, feedback text	account_id

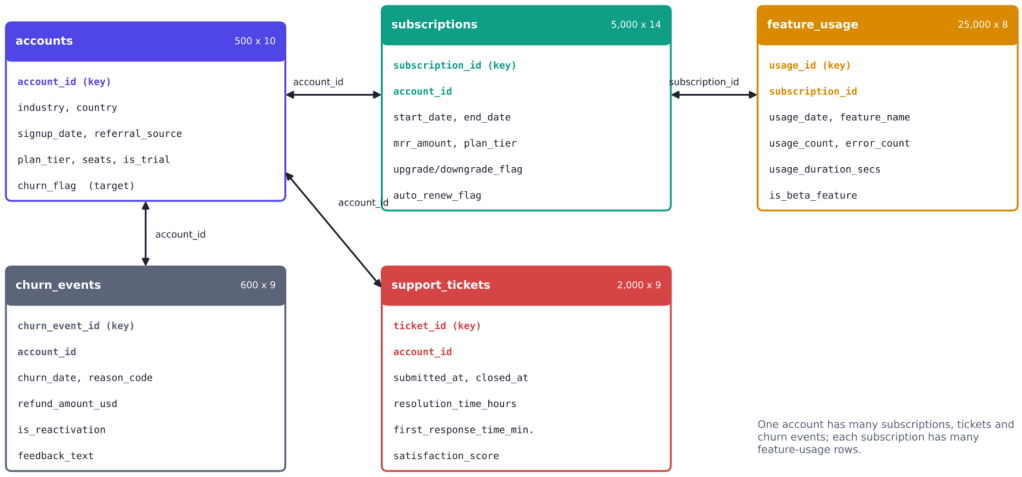


Fig. 1. Relationships between the five source tables. One account has many subscriptions, support tickets and churn events; one subscription has many feature-usage rows.

2.3 Data Quality

The cleaned tables contain no duplicate rows. Missing values occur in a small number of columns (Table 2). Missing end dates are expected for subscriptions that are still active; missing satisfaction scores and feedback text reflect optional fields.

Table 2. Missing values in the cleaned source tables.

Column	Missing values	Interpretation
subscriptions.end_date	4,514 of 5,000	Open (still active) subscriptions
support_tickets.satisfaction_score	825 of 2,000	Score not provided for the ticket
churn_events.feedback_text	148 of 600	Free-text field left empty

2.4 Target Variable

The target is **churn_flag** in the accounts table: 110 accounts are churned and 390 retained (22.0% churn). The code available for this report does not show how churn_flag was derived, so it is used as supplied. The churn_events table does not map one-to-one onto the flag: it covers 352 accounts, but only 110 accounts are flagged as churned. This matters for feature construction (Section 4.3) and for the limitations (Section 9).

3 System Architecture and Workflow

The project is organised as four notebooks and one application file. Notebook 01 cleans the raw tables, notebook 02 explores them and builds the account-level dataset used by the dashboard, and notebooks 03 and 04 contain the classification work. Notebook 04 is the final modelling run and the source of every model result in this report; notebook 03 is an earlier iteration (80/20 split) that it supersedes. The Streamlit application reads the dashboard dataset and the model artifacts saved by notebook 04.

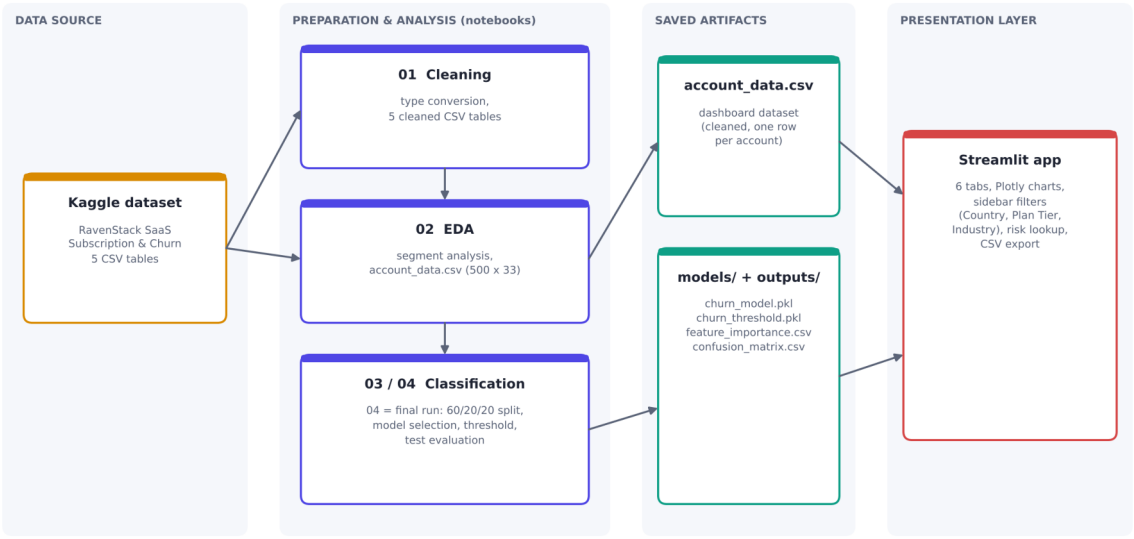


Fig. 2. System architecture: from the Kaggle tables to the Streamlit dashboard.

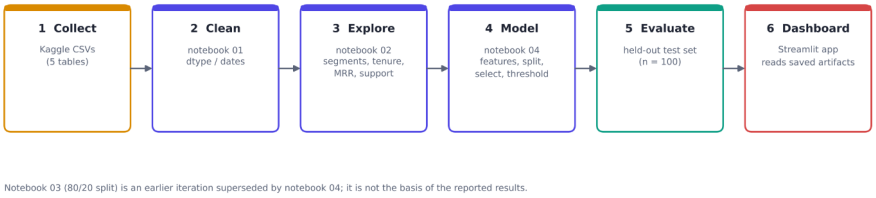


Fig. 3. Project workflow.

Table 3. Technology stack.

Component	Tools (verified from the project files)
Data handling and notebooks	Python 3.14.2, pandas, NumPy, matplotlib, seaborn
Modelling	scikit-learn 1.8.0 (Pipeline, ColumnTransformer, GradientBoostingClassifier, RandomForestClassifier, LogisticRegression), joblib 1.5.3
Dashboard	Streamlit, Plotly Express, custom CSS dark theme

Versions are those printed in notebook 04; the application's runtime versions are not recorded in the project files.

4 Data Preparation and Feature Engineering

4.1 Cleaning (notebook 01)

Notebook 01 converts boolean columns to 1/0, standardises date formats and writes the five cleaned tables. It does not impute or drop rows; missing-value handling is deferred to the modelling pipeline.

4.2 Account-level dataset for the dashboard (notebook 02)

Notebook 02 aggregates subscriptions, feature usage, support tickets and churn events to one row per account and merges them with the accounts table. It adds a first churn date per account, a tenure measure (signup to churn date, or to the observation end of the data if no churn event exists), and tenure groups of 0-6 months, 6-12 months and 1+ years. Structural gaps (accounts with no tickets or churn events) are filled with zero, values are rounded to two decimals, and the result is saved as account_data.csv (500 rows x 33

columns). This dataset feeds the dashboard charts. Notebook 02 also records four fields as leakage risks and excludes them from modelling: churn_date, reason_code, refund_amount_usd and feedback_text.

4.3 Model dataset and prediction date (notebook 04)

The model must not see information from after the moment at which a prediction would be made. Notebook 04 therefore defines a **prediction date** for each account and builds features only from records dated on or before it:

- **Observation end** is 2024-12-31, the latest date in the data.
- **Retained accounts (churn_flag = 0):** prediction date = observation end.
- **Churned accounts (churn_flag = 1):** prediction date = first churn-event date minus 30 days.
- Subscription, usage and ticket records are filtered to dates on or before the prediction date before aggregation; tenure is measured at the prediction date.

This produces a modelling table of 500 rows with 19 features (Table 4) and the target. Churned accounts that have no row in churn_events (35 of the 110) receive no prediction date, and therefore no behavioural features; this has consequences described in Section 9.

Table 4. The 19 model features.

Group	Features
Account profile	plan_tier, industry, country, seats, is_trial, tenure_months
Subscription	avg_mrr, upgrade_count, downgrade_count, auto_renew_count
Product usage	total_usage, total_usage_duration, total_errors, beta_feature_usage
Support	total_tickets, avg_resolution_hours, avg_response_minutes, avg_satisfaction, escalated_tickets

5 Exploratory Analysis

Table 5 lists the descriptive findings shown in the dashboard. All values were checked against account_data.csv. They describe associations within this dataset; group sizes are sometimes small (for example, Germany has 25 accounts), and none of the comparisons is tested for statistical significance.

Table 5. Verified descriptive findings.

Dimension	Finding
Overall	110 of 500 accounts churned (22.0%); average MRR $2,264.18
Plan tier	Pro 21.9%, Basic 22.0%, Enterprise 22.1% - essentially flat
Country	Highest rate: Germany 32.0% (8 of 25). Most churned accounts: United States, 68 (23.4%)
Industry	DevTools 31.0% (35 of 113)
Referral source	Event referrals 30.2% (29 of 96)
Trial status	Trial accounts 25.8% vs non-trial 21.1%
Tenure group	0-6 months 18.7%, 6-12 months 23.5%, 1+ years 31.2%
Average MRR	Retained $2,315.30 vs churned $2,082.92
Plan changes (avg. per account)	Upgrades 1.06 retained / 1.05 churned; downgrades 0.44 / 0.43
Support satisfaction (avg.)	Retained 3.95 vs churned 4.00
Refunds (avg. total)	Churned 20.63 vs retained 16.37

Reading the tenure and refund findings

Tenure is measured to the churn date for accounts that have a churn event and to the observation end otherwise. Churned accounts are therefore observed for a different span than retained ones, so the rising churn rate across tenure groups partly reflects how tenure is defined and should not be read as 'longer-tenure customers are more likely to leave'. Refund totals come from churn_events and exist only after a churn event, so they are not available as a prior signal.

6 Modelling Methodology

6.1 Data split

The 500-row modelling table was split with stratification on the target (random_state 42) into 60% train, 20% validation and 20% test. Each split keeps the same class balance.

Table 6. Stratified 60/20/20 split.

Split	Accounts	Churned	Retained	Used for
Train	300	66	234	Fitting models
Validation	100	22	78	Model comparison and threshold choice
Test	100	22	78	Final evaluation only

6.2 Preprocessing

Preprocessing is part of a scikit-learn Pipeline so that it is fitted on training data only. Numeric features use median imputation; categorical features (plan_tier, industry, country) use most-frequent imputation followed by one-hot encoding with unknown categories ignored. No feature scaling is applied. The 19 raw features become 31 processed columns.

6.3 Candidate models

Table 7. Candidate models. Hyperparameters were fixed in the notebook; no tuning search is recorded.

Model	Configuration
Logistic Regression	class_weight = balanced, max_iter = 2000, random_state = 42 (the optimiser reported a convergence warning at 2000 iterations)
Random Forest	500 trees, max_depth = 6, min_samples_leaf = 3, class_weight = balanced, random_state = 42
Gradient Boosting	150 estimators, learning_rate = 0.05, max_depth = 2, min_samples_leaf = 5, random_state = 42

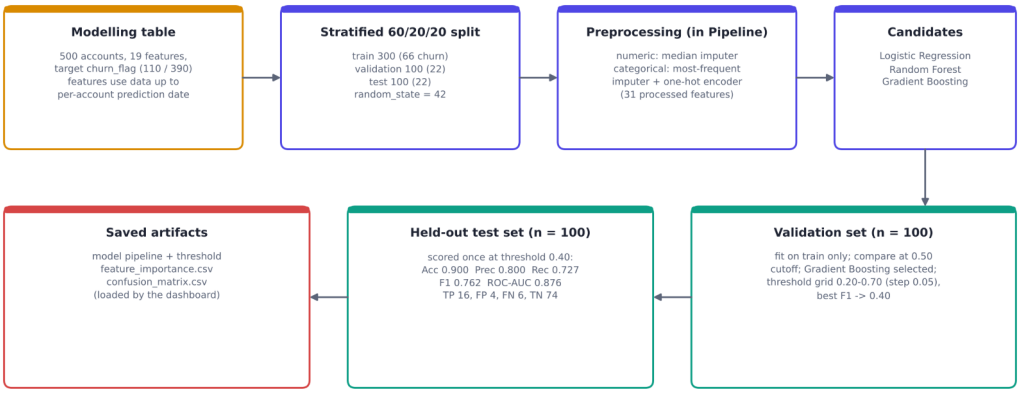


Fig. 4. Modelling pipeline in notebook 04.

6.4 Model and threshold selection

The three candidates were fitted on the training set and compared on the validation set at the default 0.50 cutoff. Gradient Boosting had the highest accuracy, precision, recall and F1 and was selected. Its decision threshold was then chosen on the validation set by scanning 0.20 to 0.70 in steps of 0.05 and taking the threshold with the highest F1. Thresholds 0.40 and 0.45 tie (F1 0.757); the notebook picks the first, 0.40. The selected model, fitted on the training split only, and the threshold were saved with joblib.

Table 8. Gradient Boosting on the VALIDATION set at each threshold (n = 100).

Threshold	Accuracy	Precision	Recall	F1
0.20	0.850	0.640	0.727	0.681
0.25	0.860	0.682	0.682	0.682
0.30	0.890	0.789	0.682	0.732
0.35	0.900	0.875	0.636	0.737
0.40	**0.910**	**0.933**	**0.636**	**0.757**
0.45	0.910	0.933	0.636	0.757
0.50	0.900	0.929	0.591	0.722
0.55-0.70	0.880	0.917	0.500	0.647

6.5 Cross-validation

As a robustness check, the Gradient Boosting pipeline was evaluated with 5-fold stratified cross-validation on all 500 rows: accuracy 0.926 (SD 0.015), precision 0.924 (0.051), recall 0.727 (0.076), F1 0.810 (0.043) and ROC-AUC 0.916 (0.033). These figures were not used for selection and use a fixed 0.50 cutoff, so they are not comparable with the test metrics.

7 Results

7.1 Validation results (model selection)

Table 9. Candidate comparison on the VALIDATION set (n = 100) at the 0.50 cutoff.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.710	0.370	0.455	0.408	0.686
Random Forest	0.850	0.818	0.409	0.545	0.810
Gradient Boosting	0.900	0.929	0.591	0.722	0.805

7.2 Final test results

The selected model was applied once to the held-out test set at the validation-chosen threshold of 0.40.

Table 10. Final model on the TEST set (n = 100, threshold 0.40).

Accuracy	Precision	Recall	F1	ROC-AUC
0.900	0.800	0.727	0.762	0.876

Table 11. Confusion matrix on the TEST set.

	Predicted retained	Predicted churned
Actual retained	74 (true negatives)	4 (false positives)
Actual churned	6 (false negatives)	16 (true positives)

Of the 22 churned accounts in the test set, the model identified 16 and missed 6; of the 20 accounts it flagged, 16 were genuine churners and 4 were not. The confusion matrix agrees with the reported precision (16/20 = 0.800) and recall (16/22 = 0.727). The notebook also printed a test classification report at the default 0.50 cutoff before the final threshold was applied (churned class: precision 0.83, recall 0.68); that output was not used to choose the model or threshold.

> **Validation versus test**
>
> Validation metrics (Table 9) were produced at a 0.50 cutoff and drive model selection. Test metrics (Table 10) use the 0.40 threshold chosen on validation and give the final estimate. The two sets contain different accounts and different thresholds, so the figures are not directly comparable; in particular the Gradient Boosting validation ROC-AUC (0.805) and test ROC-AUC (0.876) differ because they are measured on different 100-account samples, not because the model changed.

7.3 Feature importance

Table 12. Top-10 Gradient Boosting feature importances (impurity-based, training-set model).

Rank	Feature	Importance
1	Avg MRR	0.528
2	Auto Renew Count	0.152
3	Tenure Months	0.146
4	Total Usage Duration	0.044
5	Total Errors	0.027
6	Upgrade Count	0.023
7	Seats	0.020
8	Total Tickets	0.019
9	Avg Response Minutes	0.012
10	Total Usage	0.008

The top three features account for 82.6% of total importance. Impurity-based importance shows which inputs the fitted trees split on; it does not show that those inputs cause churn, and it can overstate features that proxy for data artefacts (see Section 9.2).

7.4 Uncertainty

With only 22 churned accounts in the test set, each additional churner correctly or incorrectly flagged moves recall by about 4.5 percentage points. A bootstrap resample of the test predictions (2,000 draws, supplementary analysis performed while preparing this report) gives approximate 95% intervals of 0.77-0.97

for ROC-AUC, 0.52-0.90 for recall and 0.61-0.96 for precision. The test metrics should therefore be read as indicative rather than precise.

8 Streamlit Dashboard

The dashboard (titled ChurnSight) is a single-page Streamlit application with a dark theme. A sidebar lets the reader filter by Country, Plan Tier and Industry; a header strip shows four KPI cards; and six tabs present the analysis. Charts are built with Plotly Express. Each tab ends with an insight box. These insight boxes and the recommendation cards are written text in the application code, not generated from the data at run time, and they do not change when filters change; their numbers were checked against the dataset for this report.

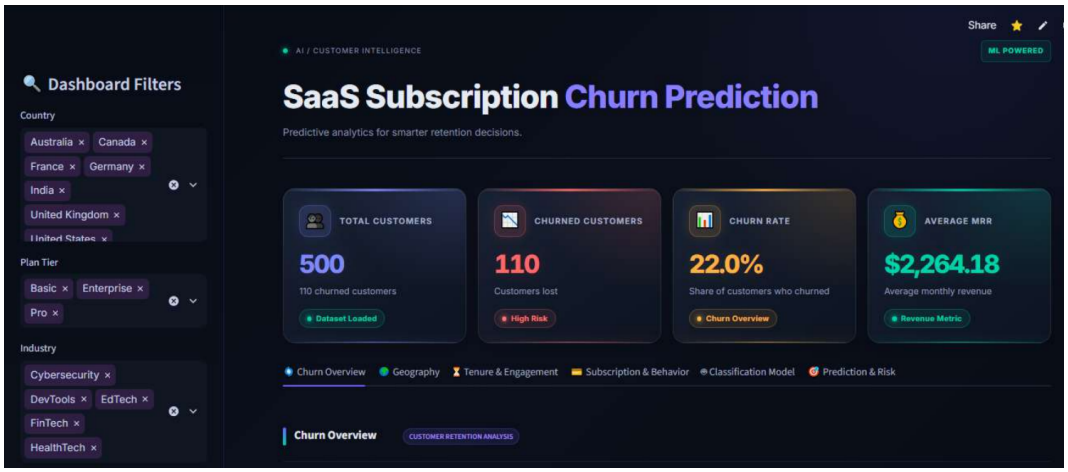


Fig. 5. Dashboard header: KPI cards (500 customers, 110 churned, 22.0% churn rate, $2,264.18 average MRR), tab bar and sidebar filters.

8.1 Churn Overview

A donut chart shows the retained/churned split (78% / 22%) and a bar chart shows churn rate by plan tier. The insight box states that churn is consistent across plans at about 22%, suggesting plan tier alone is not a key differentiator.



Fig. 6. Churn Overview tab.

8.2 Geography

Churn rate by country and a country-by-plan heatmap. The insight box notes Germany's highest rate (32%, 8 of 25) and the United States' largest churned count (68). Several country-plan cells rest on very few accounts.

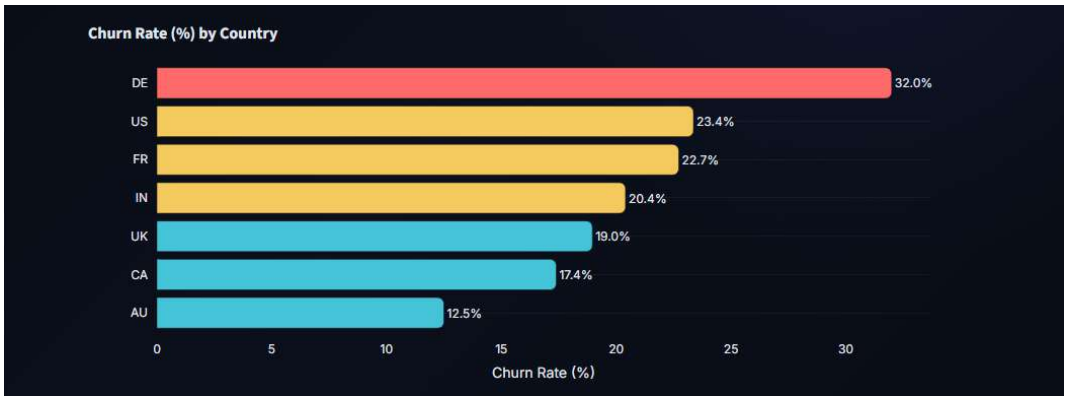


Fig. 7. Churn rate by country.



Fig. 8. Churn rate by country and plan tier, with the Geographic Insight box.

8.3 Tenure and Engagement

Churn rate by tenure group and a selectable behaviour metric (beta-feature usage, usage duration or support satisfaction). The tenure insight reports 31.2% churn for 1+ years versus 18.7% for the newest group; see the note in Section 5 on how tenure is defined.



Fig. 9. Churn rate by tenure group.



Fig. 10. Support satisfaction by churn status (3.95 retained, 4.00 churned).

8.4 Subscription and Behaviour

Average MRR by churn status ($2,315.30 retained, $2,082.92 churned) and average upgrades and downgrades per account. Plan-change counts are almost identical for the two groups.



Fig. 11. Average monthly recurring revenue by churn status.



Fig. 12. Subscription changes by churn status, with the Subscription Behaviour Insight box.

8.5 Classification Model

The tab shows five KPI cards for the final model, a validation-set comparison chart for the three candidates, the test-set confusion matrix and the top-10 feature importances. The KPI cards (0.900 / 0.800 / 0.727 / 0.762 / 0.876) are the held-out **test** results at threshold 0.40; the comparison chart is the **validation** comparison at a 0.50 cutoff, as its caption states. The card values and the validation values are fixed numbers written in the application code; the confusion matrix and feature importance are read from files saved by notebook 04.



Fig. 13. Classification Model tab: final-model KPI cards (test set) above the validation-set candidate comparison.

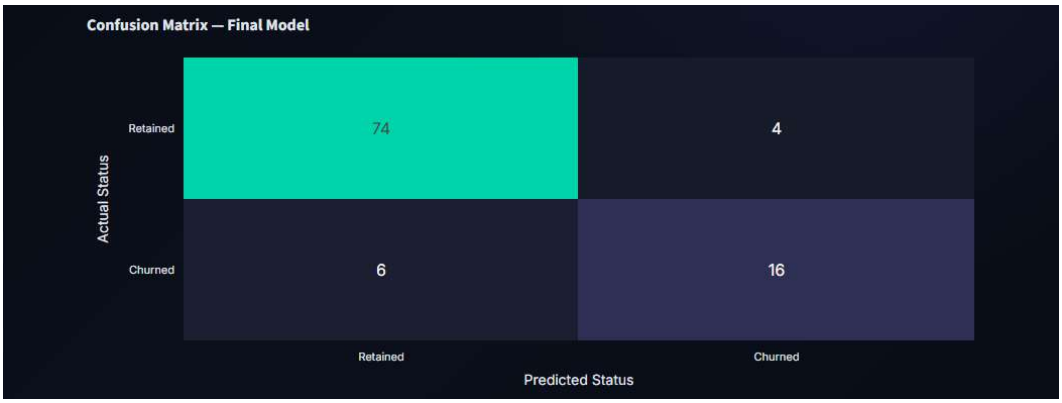


Fig. 14. Confusion matrix of the final model on the test set.



Fig. 15. Top features influencing churn, with the Model Performance Insight box.

Recommendation cards

Four static cards summarise the segments highlighted above (DevTools, refund patterns, trial conversion, acquisition source). Each pairs a finding with a suggested next analysis and is worded as something to investigate rather than a conclusion.

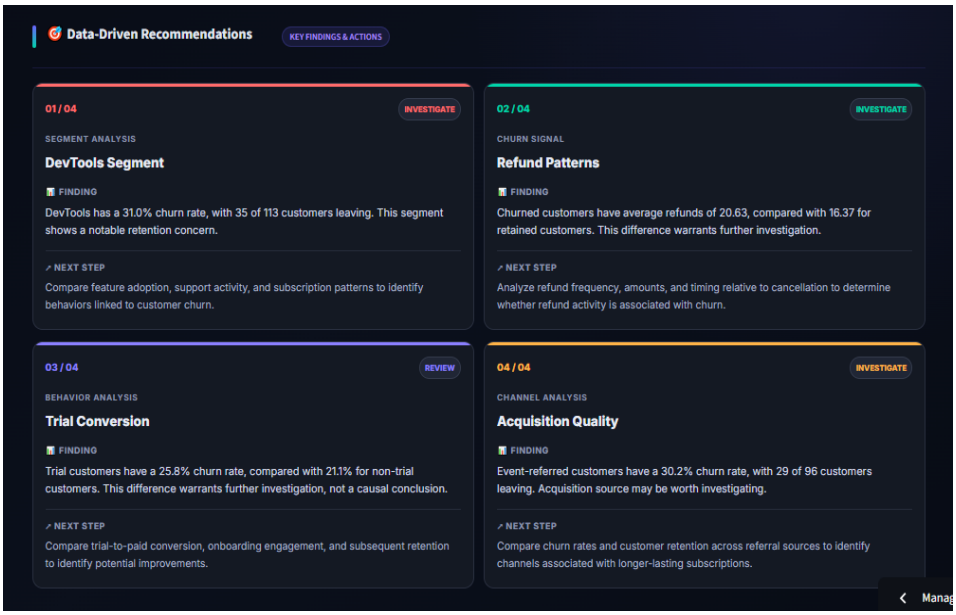


Fig. 16. Data-driven recommendation cards.

8.6 Prediction and Risk

The reader selects an account; the application passes that account's row from the dashboard dataset to the saved model and shows the churn probability, a risk band and a plain-language message. Bands are HIGH for probability of 0.60 or more, MEDIUM from the saved decision threshold (0.40) up to 0.60, and LOW below 0.40. The tab also provides a CSV download of the cleaned dataset. The two examples below are individual lookups, not performance evidence (see Section 9.3).

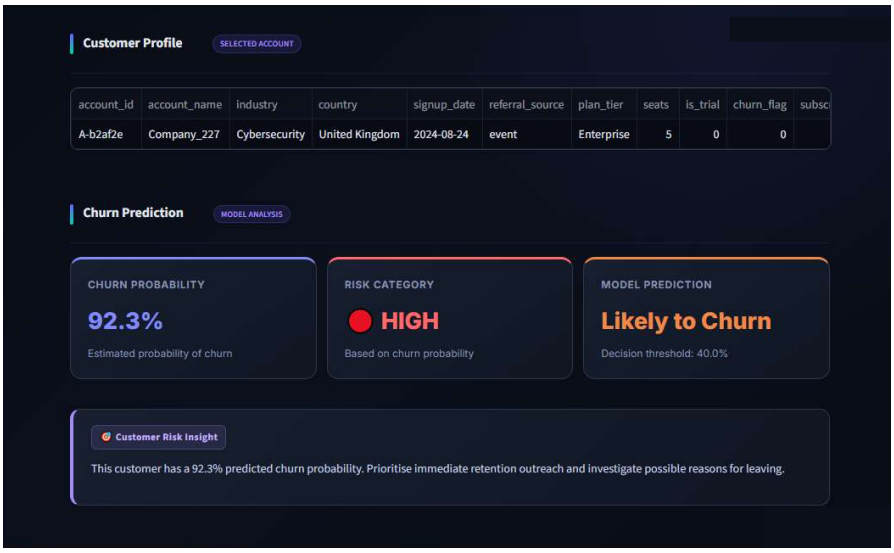


Fig. 17. Prediction & Risk: an account scored HIGH (92.3%).

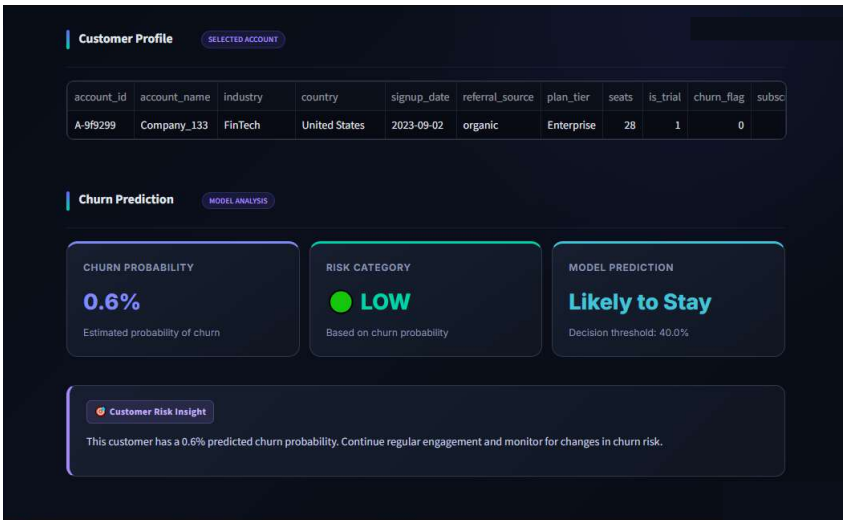


Fig. 18. Prediction & Risk: an account scored LOW (0.6%).

9 Limitations and Validity

The items below were identified while documenting the project. They do not change the numbers reported above, which are the project's own results; they bound how far those numbers can be trusted.

9.1 Small test set

The test set holds 100 accounts and 22 churners. Metrics move several points with a single account (Section 7.4), and the model was compared on a single validation split of the same size. Cross-validation on all 500 rows (Section 6.5) points the same way but is a separate estimate.

9.2 Missing features are informative

35 of the 110 churned accounts have no churn_events row, so they receive no prediction date and all behavioural features are missing; a further 27 churners have no subscription starting before their prediction date. In total, **avg_mrr is missing for 62 of 110 churned accounts and for none of the 390 retained accounts.** The pipeline fills these gaps with the median, which turns 'avg_mrr was missing' into a value the trees can isolate. In the test set, 11 of the 22 churners have missing avg_mrr, and a rule that flags only those accounts would already reach precision 1.0 and recall 0.50. On the 89 test accounts with a real avg_mrr, the final model's ROC-AUC is about 0.76 and its recall about 0.45 (supplementary reproduction from the cleaned CSVs, which matched the notebook's test metrics and confusion matrix exactly). The headline test recall of 0.727 therefore partly reflects this data pattern, and the high importance of avg_mrr (Table 12) may partly reflect it as well. The notebooks do not include an ablation that separates the two effects.

> **Why this matters**
>
> Missingness of avg_mrr and the other behavioural features is determined by how prediction dates are built for churned accounts, not by customer behaviour. A deployed model would not see this pattern for new accounts. The reported metrics should be quoted with this caveat.

9.3 Dashboard scoring uses different features from training

The model was trained on features restricted to the period before each account's prediction date. The Prediction & Risk tab instead feeds each account's whole-history features from account_data.csv, in which gaps were filled and no date cut-off applies. In addition, about 60% of accounts were training rows, so probabilities shown for them are not held-out predictions. The tab illustrates how a scored account would be presented; it is not evidence of predictive performance. In a supplementary check, only three accounts in the test split reached the HIGH band on dashboard features, and none of them churned.

9.4 Other limitations

- **Label definition.** How churn_flag was derived is not visible in the code, and many accounts with churn events (including reactivations) are not flagged as churned. The label is used as supplied.
- **Data quality.** Sixteen churned accounts have negative tenure at their prediction date (a prediction date earlier than signup), which suggests inconsistent dates in the synthetic data.
- **Synthetic data.** Results describe the RavenStack dataset and may not transfer to real customer bases.
- **Limited tuning.** Hyperparameters were set once; no search or calibration is recorded. Logistic Regression did not fully converge at 2,000 iterations and features were not scaled, so its weak validation result may understate what a tuned linear model could achieve.
- **Static text in the app.** Insight boxes, recommendation cards, model KPI cards and the validation comparison are hard-coded; if the model or data change they must be updated by hand.
- **Earlier iteration.** Notebook 03 chose a threshold on its test set and refit on all rows; its very high test scores are not valid estimates and are excluded from this report.
- **Saved model files.** The .pkl files were not part of the material reviewed. Notebook 04 shows the code that saves the model (fitted on the training split) and the threshold; the saved files themselves were not inspected.

10 Conclusion and Future Work

The project delivers a complete workflow: five raw tables are cleaned, combined to one row per account, explored, modelled and presented in a six-tab dashboard. Within the dataset, plan tier is not associated with churn, while some segments (DevTools, event-referred accounts, trial accounts, Germany) show higher churn rates and warrant closer inspection. The final Gradient Boosting model reaches test accuracy 0.900 and ROC-AUC 0.876 at threshold 0.40, with recall of 0.727 on 22 churned test accounts, subject to the caveats in Section 9.

Natural next steps, none of which has been done, are:

- Handle accounts with no churn event explicitly, for example by assigning prediction dates independently of the label, and re-evaluate with and without missing-indicator features.
- Score the dashboard with features built the same way as the training features, or label the tab as illustrative.
- Use repeated or nested cross-validation and report confidence intervals, given the small number of churners.
- Generate the dashboard's model figures from the saved test_metrics.csv and validation results instead of hard-coding them.
- Validate the approach on a real, non-synthetic subscription dataset.

References

[1] River @ Rivalytics, "SaaS Subscription and Churn Analytics Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/rivalytics/saas-subscription-and-churn-analytics-dataset

[2] Nandhusri05, "RavenStack-SaaS-Analytics," GitHub repository (README describes the dataset as fully synthetic). [Online]. Available: https://github.com/Nandhusri05/RavenStack-SaaS-Analytics

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, 2001.

[5] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[6] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861-874, 2006.

[7] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56-61.

[8] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[9] Plotly Technologies Inc., "Plotly Python graphing library." [Online]. Available: https://plotly.com/python/

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90-95, 2007.

Online sources accessed October 2026.

Appendix A Verification Status

Status of the main technical statements in this report. 'Verified' means confirmed from the notebooks, application code or project data files; items that could not be confirmed say so.

Item	Status	Basis
Notebook 04 is the source of final model results	Verified	Code and printed outputs in notebook 04
Test metrics 0.900 / 0.800 / 0.727 / 0.762 / 0.876 and confusion matrix [[74,4], [6,16]]	Verified	Notebook 04 outputs; independently reproduced from the cleaned CSVs with the same split and settings
Split sizes and class counts (300/100/100; 66/22/22 churners)	Verified	Notebook 04 outputs
Threshold 0.40 chosen on validation by best F1	Verified	Notebook 04 threshold table
Candidate validation metrics (Table 9)	Verified	Notebook 04 printed table
Dashboard statistics and insight-box numbers	Verified	Recomputed from account_data.csv
Dashboard model KPI cards and validation chart are hard-coded	Verified	Application code
Contents of churn_model.pkl / churn_threshold.pkl as deployed with the app	Not verified from the available project files	Files not provided; notebook code saves them
Dataset is synthetic	Not verified from the Kaggle listing itself	Stated in a public repository README [2]
How churn_flag was derived	Not verified from the available project files	Not in notebook code
Hyperparameter tuning	None evidenced	Single fixed configuration in notebook 04
Application runtime library versions	Not verified from the available project files	Only notebook environment versions are printed
Production deployment, real-time scoring, alerts	Not claimed	Out of scope